The Documentation Dilemma

Time & Consistency

Manual documentation is slow, often inconsistent, and takes developers away from core coding tasks. This leads to outdated or missing docs.

Critical Security Risk

Using cloud-based AI tools (like ChatGPT) is not an option. It requires sending proprietary source code to third-party servers, creating an unacceptable IP risk.

Lack of Context

Standard documentation records *what* changed, but rarely *why*. New developers have no way to understand the project's historical narrative or the intent behind past changes.

Proposed Solution: A Secure On-Premise Assistant

- **100% Secure & Offline:** An on-premise application that runs entirely on our own hardware. It is "air-gapped," meaning no data, source code, or IP ever leaves the network.
- Runs on a local Ubuntu server.
- Uses a local AI model (no external API calls).
- Provides intelligent, context-aware documentation, securely.

Design Pillar: 100% Offline Security

This is the foundation of the project. The entire application stack—from the web server to the AI model itself—is self-hosted. It is physically impossible for data to leak.

The system is comprised of three parts, all running locally:

1. A Python-based web server.
2. A local, file-based database for memory.
3. A local AI model loaded directly into the server's memory.

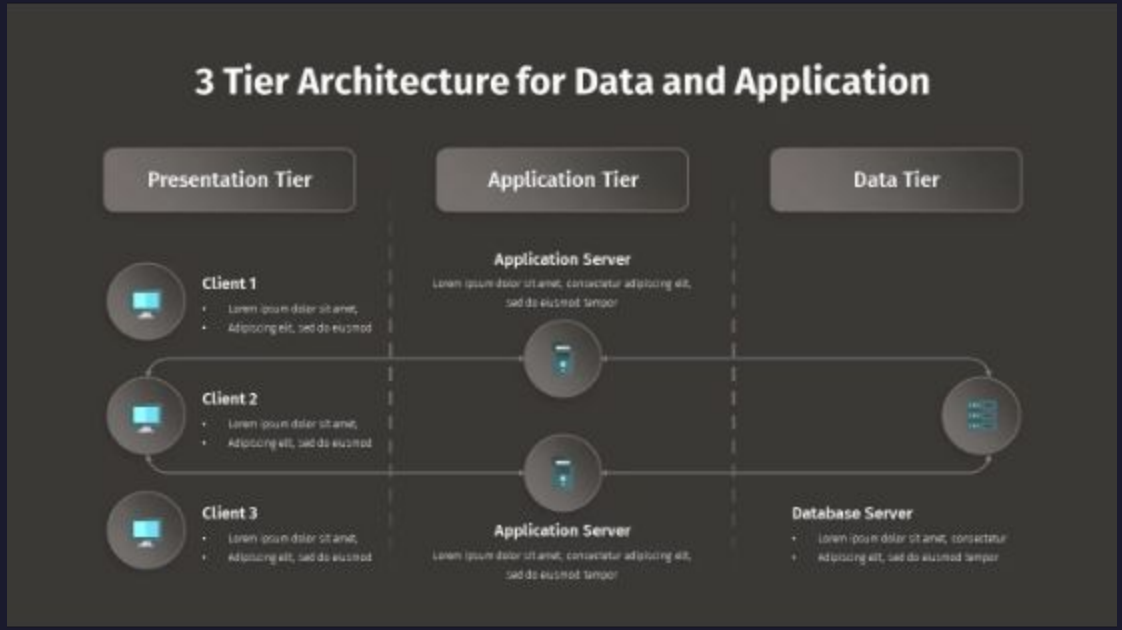
.

System Architecture Overview

Simple & Robust 3-Tier Model

The application is built on a classic, robust 3-tier architecture, all running on a single Ubuntu host.

- **Frontend (UI):** A simple HTML/JS page served to the user's browser.
- **Backend (Logic):** A Python Flask server that handles requests and orchestrates tasks.
- **Data/AI (Engine):** The local file system, Git, SQLite DB, and the Local LLM.

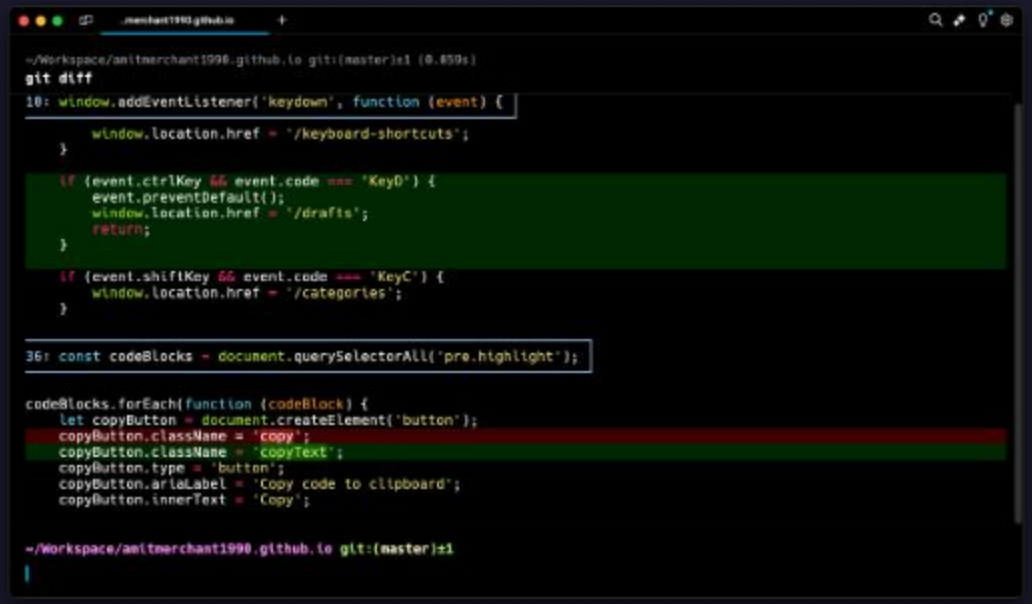


How It Works (Step 1): Git Analysis

Reading the `git diff`

The assistant doesn't just read the current code; it analyzes the *changes*. When a user requests documentation, the backend uses the GitPython library to:

1. Find the latest commit in the project.
2. Identify its parent commit.
3. Extract the raw `git diff` (lines added, removed, or changed).
4. Extract the developer's commit message.



How It Works (Step 2): Historical "Memory"

The "Secret Sauce"

This is the system's key feature. Before analyzing the new diff, the assistant queries its internal **SQLite database**.

It searches for the documentation *it wrote* for the *previous* commit (the parent commit). This "memory" provides the historical context that all other tools lack, allowing the AI to build a narrative over time.

How It Works (Step 3): Dynamic Prompt Engineering

[PAST CONTEXT]

(From the SQLite Database)

"Last time, you wrote: 'Added the initial UI framework for the login page, including an Email and Password field. Button is not functional.'"

[NEW DATA + TASK]

(From Git & User)

"Now, this new 'git diff' adds a 'Login Button' and a 'validate_user' function...

TASK: Write a bulleted list for the 'What's New' section."

The "Memory" Advantage: An Example

Without Memory (Other Tools)

"Added a login button."

"Added a validation function."

(Accurate, but context-free and not very useful.)

With Memory (Our Tool)

"Completed the user login workflow by:

- Activating the 'Login Button'.
- Connecting it to the new 'validate_user' function.

This builds on the UI framework from the previous **commit**."

(Intelligent, contextual, and truly valuable.)

Core Technology: Backend

Flask

A lightweight and robust Python framework. It serves the simple HTML frontend and provides the core API endpoint (/api/generate_docs).

GitPython

A powerful Python library that provides a clean, object-oriented interface for interacting directly with Git repositories to read commits, branches, and diffs.

SQLite

A serverless, file-based SQL database engine. It's built into Python and provides the perfect, lightweight solution for storing the "memory" of past summaries.

Core Technology: The AI Engine

`llama-cpp-python`

The core of the AI engine. This library provides high-performance Python bindings to run LLMs directly on local hardware (CPU or GPU), allowing for fast, secure, offline inference.

Mistral 7B (GGUF)

The AI model itself. We use a quantized (GGUF) version of Mistral 7B, an open-source model that offers a world-class balance of intelligence, speed, and small file size.

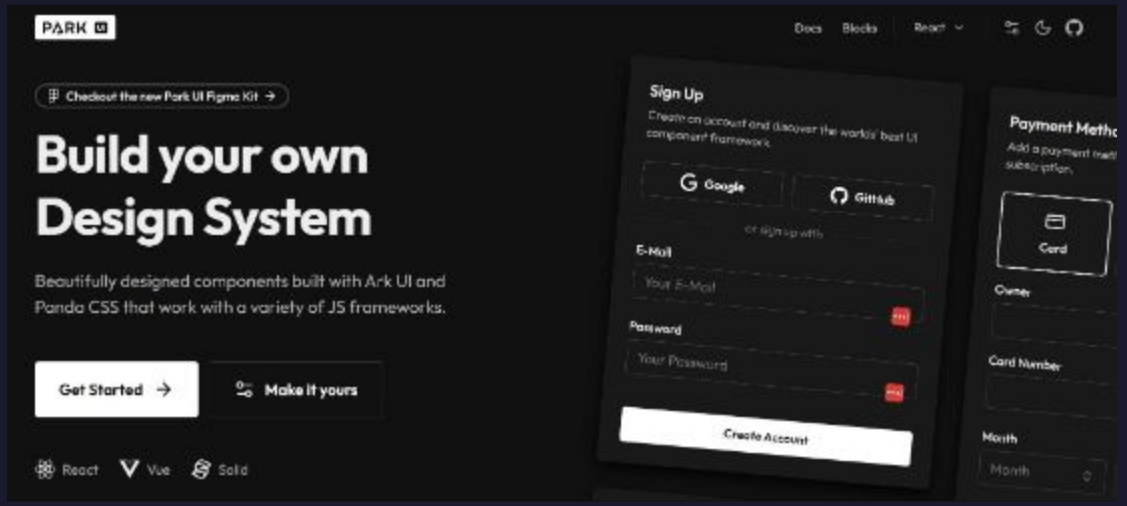
The User Interface

A Tool, Not a Chatbot

The UI is a simple, clean, and purposeful webpage. The workflow is straightforward:

1. User provides the **Project Path** on the server.
2. User provides a **Format Prompt** (e.g., "Write a bullet list," "Write a formal paragraph").
3. User clicks "Generate."

The backend handles the rest, returning the new, context-aware documentation.



Project Impact & Benefits

- **Zero IP Risk:** Guarantees 100% security and confidentiality for proprietary source code. This is the primary mission success.
- **High-Quality, Consistent Docs:** Creates reliable, intelligent, and consistent documentation for every single update, standardizing our process.
- **Knowledge Preservation:** The AI's "memory" becomes the project's institutional memory. New developers can instantly understand the *history* and *intent* of the code.
- **Developer Efficiency:** Frees senior developers from the time-consuming task of writing documentation, allowing them to focus on building and innovation.

Roadmap

1

Core Engine: Git analysis, AI inference, and historical "memory" for the latest commit.

2

Full Repo Scan: Ability to analyze *all* past commits to generate a complete project history document from scratch.

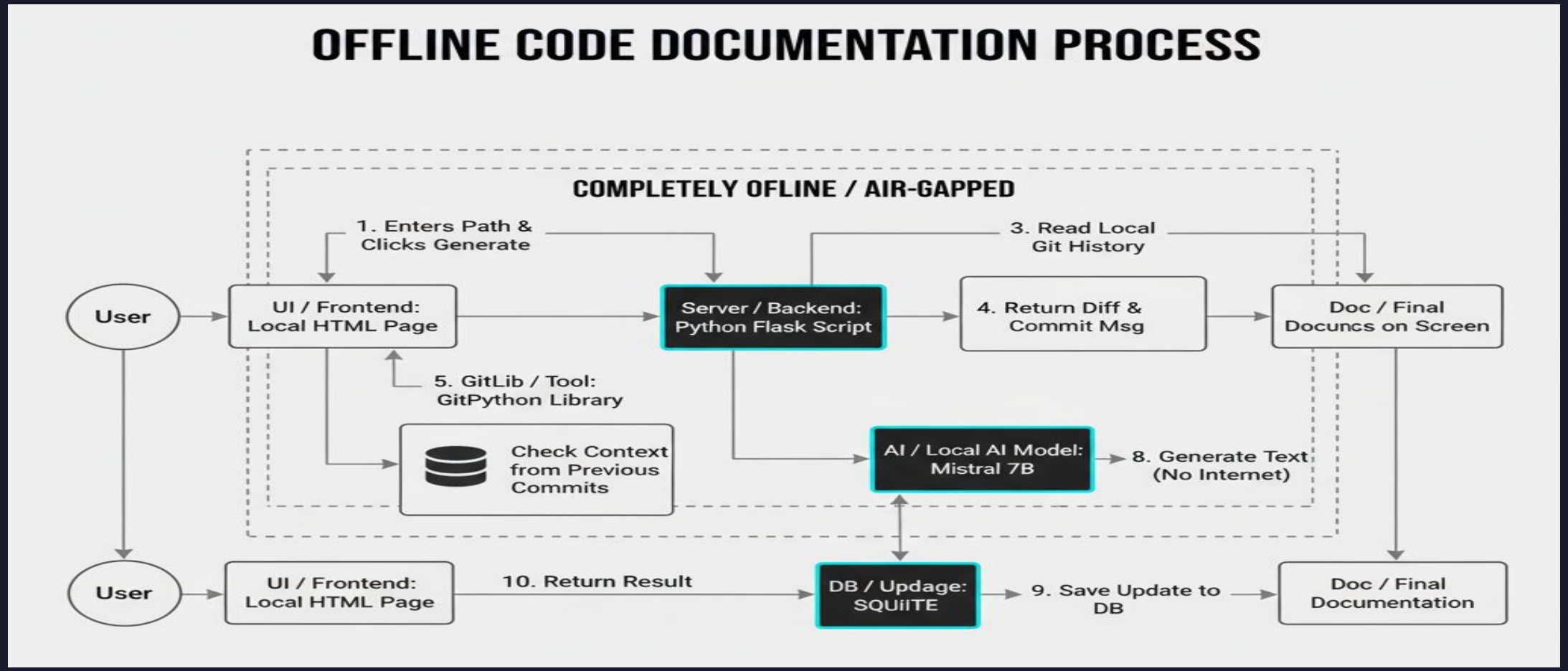
3

Codebase Q&A: Enable "chat with your code" to ask questions like "Where is the auth logic handled?".

4

IDE Integration: Move from a web UI to a dedicated plugin within Qt Creator/VS Code for a seamless developer workflow.

Model Selection: Why Mistral 7B?

Mistral 7B (GGUF quantized) provides the optimal balance. It's small enough (~4GB) to run on local developer hardware, is extremely fast for inference, and has reasoning capabilities that are more than sufficient for summarizing code diffs, outperforming larger models in its class.

Complete Data Flow